ORDI: Orbit-Aware Redundant Distributed Inference for LEO Earth Observation Constellations

An-Che Liang, Guan-Ming Chiu, Pei-Yu Wu
National Taiwan University

***Abstract*—Low-Earth-orbit (LEO) Earth observation (EO) constellations capture far more imagery than their downlink budgets can deliver, motivating onboard inference that distills raw images into compact results. Distributing inference across neighboring satellites can meet tight deadlines, but the orbital environment is hostile to distributed computing: inter-satellite links appear and disappear with orbital geometry, commercial off-the-shelf (COTS) payloads throttle under thermal and battery stress, and failures are frequently correlated across an orbital plane. We present ORDI, an orbit-aware scheduler that assigns image tiles to helper satellites with *selective* redundancy: a backup replica is placed only when its marginal reliability gain, priced through a time-decaying utility model, exceeds its energy and bandwidth cost, and backups are constrained to fault-disjoint helpers and aggregators. ORDI couples a time-expanded contact graph with per-satellite compute, battery, and thermal state, and replans within an epoch upon helper failure or straggling. We evaluate ORDI in a simulator calibrated with in-orbit measurements of a COTS payload from the Tiansuan constellation, against eight baselines spanning bent-pipe downlink, onboard-only, single-helper offloading, and full replication. Across fault intensities up to 50%, ORDI holds the deadline miss ratio to 15.7% while a non-redundant scheduler degrades to 52.4%; relative to full replication it delivers a comparable miss ratio with 30% fewer replicas and 32% less energy. Under correlated plane-wide outages, plane-disjoint backup placement matches full replication's resilience at a fraction of its redundancy, and the greedy scheduler matches an ILP reference on delivered utility while using $2.6\times$ less inter-satellite traffic.**

***Index Terms*—LEO constellations, satellite edge computing, distributed inference, fault tolerance, scheduling**

Project Category: ■ **A** (Self-chosen topic) □ **B** (AI-agent code optimization)

I. Introduction

Earth observation constellations now produce imagery at rates that overwhelm their ground segments. A single satellite can capture tens of gigabytes per orbit, yet sees a ground station for only a few minutes per pass, and smaller operators often lease a handful of stations concentrated at high latitudes. The result is a widening gap between what is sensed and what can be delivered before it loses value: a wildfire alert is useful within minutes, not hours [1], [2].

Onboard inference attacks this gap at the source. Instead of downlinking raw images, satellites run compact neural networks (e.g., MobileNetV2, YOLOv5-nano) and downlink only labels, bounding boxes, or masks, a reduction of three to four orders of magnitude in volume [3]. Recent in-orbit experiments on the Tiansuan constellation demonstrate that COTS accelerators can run such models in orbit, but also document the operational reality: computing capacity is throttled by thermal limits and battery state, and the energy available per orbit is scarce and variable [4].

A natural next step is to *distribute* inference: a source satellite partitions an image into tiles and offloads tiles over inter-satellite links (ISLs) to less-loaded neighbors, meeting deadlines that onboard compute alone cannot. Prior systems explore single-satellite query bifurcation [2] and multi-satellite placement [5], but largely assume that scheduled helpers and links remain available. In orbit this assumption routinely fails, and it fails in structured ways: an ISL outage severs all routes through a plane, a plane-wide bus anomaly takes down every satellite sharing it, and thermal throttling turns a scheduled helper into a straggler mid-task. Classic terrestrial answers, replicate everything [6] or apply MDS-coded redundancy [7], [8], translate poorly: every replica costs scarce ISL bandwidth and battery energy, and coding assumes independent failures that orbital geometry violates.

This paper asks: *how much redundancy does distributed orbital inference actually need, and where should it be placed?* Our answer is ORDI (Orbit-aware Redundant Distributed Inference), a rolling-horizon scheduler built on three ideas.

(1) Orbit-aware feasibility. ORDI plans on a time-expanded contact graph derived from orbital propagation, jointly with per-satellite state: compute rate, battery energy, temperature, queue, and availability. A candidate assignment of a tile to helper i with aggregator a is feasible only if its end-to-end latency, transfer to the helper, compute, relay to the aggregator, and downlink, fits the remaining deadline budget under current link and node state.

(2) Selective, fault-disjoint redundancy. Each tile receives one primary replica chosen by marginal utility, and at most one backup, placed only when the marginal gain in delivery probability, discounted by a freshness decay and priced against energy, bandwidth, and an explicit replication penalty λ_R, is positive. Backups must use a different helper and a different aggregator than the primary, and under correlated-failure threat models, a different orbital plane, so that a single structural failure cannot eliminate both replicas.

(3) Reactive replanning. Detected helper failures, missed contacts, and stragglers trigger re-scheduling of affected tiles within the epoch, using the same feasibility machinery.

We implement ORDI and eight baselines in a discrete-epoch simulator with a realistic LEO-EO configuration: a Walker constellation at 550 km, field-of-view-constrained task

arrivals, per-workload log-normal deadlines reflecting EO service tiers [9], and satellite power, thermal, and compute parameters loaded directly from the public in-orbit measurement logs of a Tiansuan COTS payload [4]. Experiments over 30 random seeds show that ORDI (i) dominates non-distributed baselines on deadline miss ratio and delivered utility, (ii) degrades gracefully under fault intensities up to 50% where a non-redundant scheduler collapses, (iii) matches full replication under correlated plane outages with 30% fewer replicas and 32% less energy, and (iv) tracks an ILP reference on delivered utility while using $2.6\times$ less ISL traffic.

II. Background and Motivation

A. The Downlink Bottleneck and Onboard Inference

EO satellites are bandwidth-starved. Ground contact windows last roughly 5–10 minutes per pass, and many operators schedule only polar or high-latitude stations, so a satellite over the southern hemisphere may wait most of an orbit before any downlink opportunity. Onboard inference reduces a megabyte-scale image tile to a kilobyte-scale result, shifting the bottleneck from the downlink to onboard compute, which in turn is constrained by the power and thermal envelope of a small spacecraft. In-orbit measurements show COTS accelerators operating at a fraction of their nameplate throughput under these constraints [4].

B. Why Distribute, and Why Redundantly

Tiling makes inference embarrassingly parallel: a 4×4 grid over a scene yields 16 independent units of work that ISLs can spread across a neighborhood of satellites. But distribution adds failure modes. Each replica's success requires the source to survive and emit the tile, every ISL hop on the path to hold, the helper to remain available through compute, and the aggregator's downlink contact to materialize. These events are not independent: satellites in one orbital plane share geometry, eclipse cycles, and often a deployment batch, so plane-wide outages disable many helpers at once. Full replication buys safety at roughly double the energy and ISL traffic, which a 20–30 Wh-class spacecraft cannot routinely afford. The design space between zero redundancy and full replication, priced by orbital state, is the territory ORDI explores.

III. System Model

A. Network and Contact Model

We consider a constellation $\mathcal{S}$ of N satellites and a set $\mathcal{G}$ of ground stations. Time is divided into epochs of length Δ (60 s). From orbital propagation we compute contact windows: for each satellite pair, an ISL contact when their range is below a threshold, and for each (satellite, station) pair, a downlink contact when elevation exceeds a mask. Contacts are compiled into per-epoch graphs $E(t)$; multi-epoch transfers are evaluated on the time-expanded graph by an earliest-arrival Dijkstra search, yielding $\ell_{ab}(t,d)$, the earliest time data of size d originating at node a at epoch t reaches node b, including store-and-forward waiting across epochs.

B. Satellite State

Each satellite i carries state $\sigma_i(t) = (C_i, B_i, \Theta_i, Q_i, A_i)$: compute rate (GFLOP/s), battery energy (J), temperature, queued work, and availability. Battery follows an energy balance of solar harvest against idle, compute, and communication draw; temperature follows a first-order RC model driven by compute and communication power. Availability $A_i = 0$ when battery falls below a floor, temperature exceeds a ceiling, or a failure is injected; thermal stress throttles C_i before forcing unavailability. Rather than assuming datasheet values, we instantiate σ_i parameters (compute rate, idle and active power, battery capacity, solar harvest, thermal limits) from public in-orbit measurement logs of an Atlas 200DK payload on the Tiansuan BUPT-1 satellite [4].

C. Task and Tile Model

Tasks arrive as a Poisson process, gated by sensing geometry: a task is generated only when a satellite's camera footprint covers a ground target. Task k released at r_k on source satellite s_k carries an image partitioned into tiles v with input size d^{in}_{kv}, output size d^{out}_{kv}, compute demand c_{kv} (FLOPs), and utility u_{kv}. We model four EO workloads profiled on MobileNetV2, YOLOv5-nano, a lightweight U-Net, and a Siamese CNN (wildfire detection, ship detection, cloud filtering, change detection), with per-type deadlines drawn log-normally (σ=0.6) around medians of 300, 600, 1200, and 480 s respectively, reflecting EO service tiers [9].

D. Reliability Model

Links and nodes succeed per epoch with probabilities π: ISL π_{isl}=0.97, downlink π_{down}=0.92 (0.70 adverse), node survival π_{node}=0.98. A replica of tile (k,v) on helper i with aggregator a succeeds with

$$p_{kvia} = \pi_{s_k}\,\pi_i \cdot \pi_{\text{path}}(s_k \to i) \cdot \pi_{\text{path}}(i \to a) \cdot \pi_{\text{down},a}. \quad (1)$$

Source survival is a single point of failure shared by all replicas of a tile, so it is factored once at the tile level: with replica set R_{kv},

$$z_{kv} = \pi_{s_k}\Big[1 - \prod_{r \in R_{kv}} \big(1 - p_r/\pi_{s_k}\big)\Big], \quad (2)$$

which avoids overcounting redundancy against a failure no replica can survive. Independence across the remaining terms is an explicit approximation; Section VI-H stresses it with correlated plane-wide faults.

E. Problem Statement

Let L_{kv} denote the earliest completion latency among a tile's replicas and $\tau_k(t)$ the remaining deadline budget. The scheduler chooses replica sets to maximize

$$\sum_{k,v} u_{kv}\, z_{kv}\, e^{-\alpha L_{kv}} \;-\; \lambda_E E \;-\; \lambda_C C \;-\; \lambda_R R, \quad (3)$$

subject to per-replica deadline feasibility $L_{kvia} \le \tau_k(t)$, helper energy budgets $B_i \ge B_{\min}$, and link capacities, where E is total helper energy, C is congestion-weighted ISL traffic, R

is the count of extra replicas, and $e^{-\alpha L}$ encodes freshness decay (α=0.002 s^{-1}). The exact problem is an integer program coupling routing, placement, and redundancy; we use it (Section VI-I) only as a small-instance reference.

IV. ORDI Design

Our code is available at https://github.com/namwoam/ordi.

A. Rolling-Horizon Scheduling

Each epoch, ORDI (i) advances satellite states, (ii) rebuilds the epoch contact graph, (iii) sorts pending tiles by urgency-weighted utility u_{kv}/τ_k, so scarce routing resources go to high-value, tight-deadline tiles first, and (iv) assigns replicas greedily per tile.

B. Feasibility and Candidate Enumeration

For tile (k, v), every available (helper i, aggregator a) pair defines a candidate with latency

$$L_{kvia} = \ell_{s_k i}(d_{kv}^{\text{in}}) + \frac{c_{kv}}{C_i(t)} + \ell_{ia}(d_{kv}^{\text{out}}) + \ell_a^{\text{down}}(d_{kv}^{\text{out}}), \quad (4)$$

feasible iff $L_{kvia} \le \tau_k(t)$ and $A_i = A_a = 1$. Two degenerate forms are always included: helper-as-aggregator (the helper downlinks its own result, no relay hop) and source self-processing, which guarantees ORDI never performs worse than onboard-only execution. Latencies in (4) reduce to three routing primitives per epoch, source-to-all, all-pairs, and all-to-ground earliest arrival, which ORDI computes once per epoch as shared single-source Dijkstra sweeps and caches as dense arrays, eliminating per-candidate graph searches.

C. Marginal-Gain Replica Selection

Each feasible candidate is scored by its marginal contribution to (3):

$$\Delta_{kvia} = u_{kv}\, p_{kvia}\, e^{-\alpha L_{kvia}} - \lambda_E \Delta E_{kvia} - \lambda_C \Delta C_{kvia}, \quad (5)$$

where ΔE sums receive, compute, and transmit energy on the helper and ΔC prices ISL bits by inverse residual link capacity. Candidates whose energy draw would push the helper below its battery floor, including energy already committed this epoch, are discarded. The best candidate becomes the primary replica.

A backup is then considered among remaining candidates with a *different helper and different aggregator*, and is admitted only if

$$u_{kv}\, \Delta z_{kv}\, e^{-\alpha L} - \lambda_E \Delta E - \lambda_R > 0, \quad (6)$$

where Δz_{kv} is the increase in (2) from adding the backup. Because Δz_{kv} shrinks as the primary's p grows, redundancy concentrates automatically on tiles whose primaries are unreliable, distant helpers, congested paths, marginal downlinks, rather than being spent uniformly. Under correlated-failure threat models, the backup is further required to reside in a different orbital plane than the primary (Section VI-H).

D. Replanning

Helper failure, missed contacts, and stragglers (a tile not returned within $1.5\times$ its expected latency) trigger replanning: failed helpers are marked unavailable and affected tiles re-enter scheduling within the current epoch. Recovery thus costs one scheduling pass, not a full horizon recomputation.

E. Complexity

With n active satellites and m pending tiles per epoch, candidate enumeration is $O(mn^2)$ array lookups after the per-epoch routing sweeps, which cost $O(n \cdot \text{SSSP})$. The ILP in contrast couples all tiles through shared capacities and is practical only at toy scale.

V. Implementation

ORDI is implemented in $\sim$6 k lines of Python. Orbit propagation uses Skyfield with SGP4; earliest-arrival search is a Numba-compiled single-source Dijkstra over flat arrays; epoch routing caches are shared between ORDI and all baselines so comparisons isolate scheduling policy, not implementation efficiency. Faults are declarative specifications resolved against the built constellation, enabling reproducible scenario sweeps. The COTS calibration layer parses the public Tiansuan measurement artifact [4] at runtime: inference throughput from the 4-thread image-classification logs, power draw from telemetry, and battery capacity from the published charge curves.

VI. Evaluation

A. Setup

Unless stated otherwise: Walker constellation, 6 planes $\times$ 6 satellites at 550 km (96 min period); horizon of 3 orbits (17,280 s, 288 epochs); two northern ground stations (Fairbanks, Greenwich), a deliberately constrained ground segment typical of high-latitude EO operations that creates genuine routing pressure for southern-hemisphere sources; field-of-view-gated Poisson arrivals at 6 tasks/orbit over 100 ground targets; ISL rate 200 Mb/s. Satellite parameters are the Tiansuan-calibrated COTS profile. The core comparison aggregates 30 random seeds; fault and sweep experiments use matched seeds across algorithms; error bars are one standard deviation.

Baselines. B1 direct downlink (bent pipe, no inference); B2 onboard-only; B3 onboard with $0.15\times$ output compression; B4 Serval-like priority bifurcation, one satellite per task [2]; B5 SECO-like multi-satellite placement without redundancy [5]; B6 full replication to $r_{\max}$ helpers [6]; B7 random-helper replication; B8 CoCoI-like MDS-coded redundancy adapted to contact windows [7], [8]. All baselines share ORDI's contact graphs, state model, and routing caches.

Metrics. Deadline miss ratio (primary), delivered utility per (3)'s first term, partial task coverage, recovery latency, ISL traffic, energy, and mean replicas per tile. Energy includes onboard compute, ISL communication, and spacecraft-to-ground transmission; the latter is charged as transmit power times airtime at the modeled 100 Mbps downlink rate, consistent with demonstrated high-rate CubeSat links surveyed by NASA [10].

TABLE I
CORE COMPARISON RESULTS, MEAN OVER 60 SEEDS.

Algorithm	Miss ratio	Utility	Replicas	Energy (J)
B1 direct	0.951	8.0	0	0.41
B2 onboard	0.273	137.2	0.73	211
B3 compress	0.273	140.0	0.73	21.9
B4 Serval-like	0.273	137.1	0.73	211
B5 SECO-like	0.217	145.9	0.78	228
B6 full repl.	0.217	156.4	1.56	452
B7 random repl.	0.217	145.9	1.57	456
B8 CoCoI-like	0.217	156.5	1.57	454
ORDI	**0.217**	**160.1**	**1.10**	**309**

Thus B1 pays to transmit raw tile inputs, while inference methods transmit compact outputs.

B. Core Comparison

Table I and Fig. 1 summarize. Direct downlink misses 95.1% of deadlines, confirming that bent-pipe delivery is not viable at this ground-segment scale. Onboard-only (B2–B4 cluster) reaches 27.3% misses, limited by source compute and source-local downlink geometry. Distributed schedulers (ORDI, B5–B8) cut misses to 21.7%. Within that group the distinctions are cost and utility: ORDI delivers the highest utility (160.1 ± 35.0) at 1.10 replicas/tile and 309 J, whereas full replication needs 1.56 replicas and 452 J (+46% energy, $3.5\times$ ISL traffic) to deliver *less* utility (156.4), the freshness discount penalizing its slower redundant paths. Selective redundancy is not merely cheaper; under (3) it is better.

C. Robustness Across Fault Types

Injecting each of seven fault classes against ORDI (Fig. 2): ISL disruption, stragglers, ground-contact misses, battery shortage, and thermal throttling leave the miss ratio statistically unchanged from the no-fault 0.9%, absorbed by rerouting, replanning, and backups. The damaging classes are those that remove compute en masse: plane outage and helper failure raise misses to 10.4%, and stragglers, while not raising misses, cost $1.6\times$ energy as replanned duplicates complete alongside slow originals.

D. Graceful Degradation Under Fault Intensity

Sweeping random-fault intensity from 0 to 50% (Fig. 3): the non-redundant B5 degrades from 0.3% to 52.4% misses, essentially linearly in faults. ORDI degrades to 15.7%, a $3.3\times$ improvement, while its replica count *falls* slightly (1.73 to 1.58) because fewer feasible backups exist under heavy faulting; the protection comes from placing the right backups, not more of them. B6 stays near 0.3% by paying 2.0 replicas everywhere, including the 65% of nominal epochs where redundancy buys nothing.

E. Scalability

From 12 to 60 satellites (Fig. 4), misses fall from 55% (a 12-satellite constellation simply lacks contact density toward two northern stations) to 0% at 60 satellites. ORDI and B5 see identical feasibility, so their miss ratios coincide; ORDI delivers 11–12% more utility at every scale through latency-aware placement and selective backups, and per-epoch runtime stays sub-second at 60 satellites.

F. Deadline Sensitivity

Sweeping the global deadline scale from 150 to 900 s (Fig. 5), ORDI beats single-satellite schedulers (B2, B4) at every point, missing 63.6% vs. 68.1% at a punishing 150 s and 15.3% vs. 19.6% at 900 s, with 17–22% more delivered utility. The replica column explains the mechanism: at 150 s ORDI averages just 0.56 replicas/tile, redundancy is rarely affordable and ORDI correctly declines it, spending its advantage on better primary placement; as slack grows, backups re-enter (1.16 at 900 s).

G. Pricing Redundancy with λ_R

Sweeping λ_R from 0 to 2 (Fig. 6) moves mean replicas from 1.50 to 0.76 while the miss ratio is flat, confirming that in this regime backups protect utility (delivered utility falls from 154.1 to 142.6 as backups vanish) rather than feasibility, and giving operators a single dial trading energy for delivery confidence. We use $\lambda_R{=}0.05$ throughout, the knee of the curve.

H. Correlated Failures

Independence (Eq. 2) is the standard analytical convenience that orbital geometry breaks. We disable one or two entire orbital planes mid-horizon (Fig. 7). With plane-disjoint backup placement, ORDI misses 2.5% (one plane) and 4.3% (two planes), statistically indistinguishable from full replication (2.4%, 4.0%) while using 1.68 vs. 1.95 replicas per tile. Structured redundancy, one backup placed against the actual failure correlation, substitutes for volume.

I. Greedy vs. ILP Optimality

On small instances solvable exactly, greedy ORDI and the ILP reference produce identical miss ratios (38.5% on a deliberately over-constrained instance) and near-identical delivered utility (76.7 vs. 75.6); the ILP, maximizing the modeled objective under modeled probabilities, spends $2.6\times$ the ISL traffic and 31% more energy on redundancy that realized execution does not reward. The greedy heuristic loses nothing measurable at a fraction of the cost, and unlike the ILP it scales.

J. End-to-End Real-World Data

As a final check that ORDI's advantage is not an artifact of the synthetic scenario generators, we re-run the core comparison with all three input drivers replaced by real measured data: real Planet Flock/SkySat orbits from CelesTrak [11] (24 satellites, RAAN-clustered into planes), real tasking *requests* from NASA FIRMS active-fire detections [12] (fire location, time, and radiative-power-derived urgency are all real; 387 in-view fires become wildfire tasks) with delivery deadlines set to bracket fielded wildfire-alert latencies [13], and real BUPT-1 platform telemetry [4] (battery and temperature) replayed

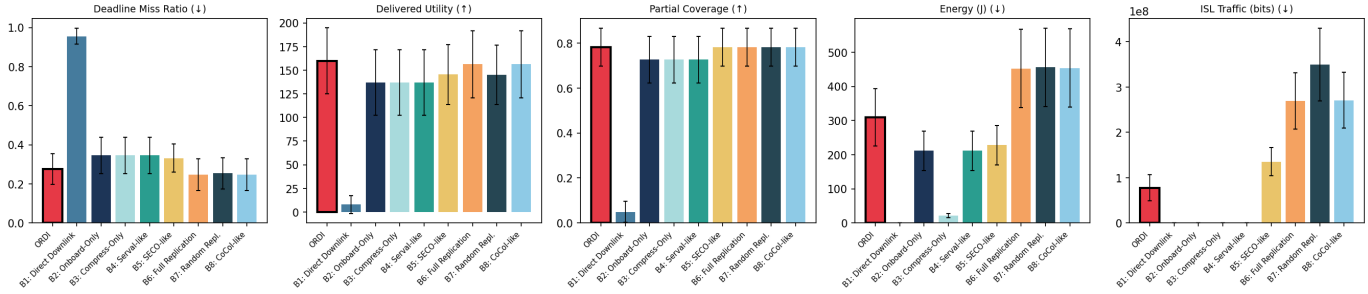

Fig. 1. Core comparison over 60 seeds (no injected faults beyond baseline stochastic link/node loss). ORDI attains the best delivered utility and ties the best miss ratio at near-minimal redundancy.

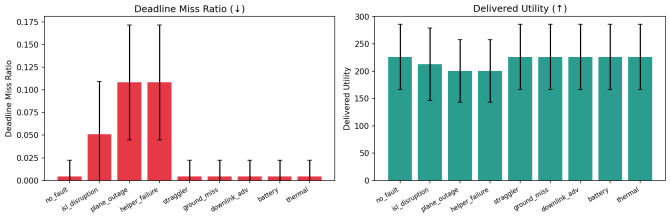

Fig. 2. ORDI robustness profile across seven fault types.

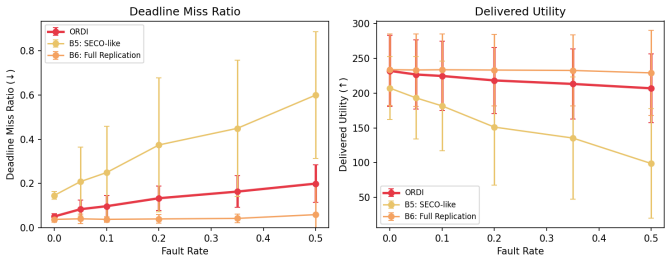

Fig. 3. Deadline miss ratio vs. fault intensity. ORDI degrades gracefully where non-redundant scheduling collapses.

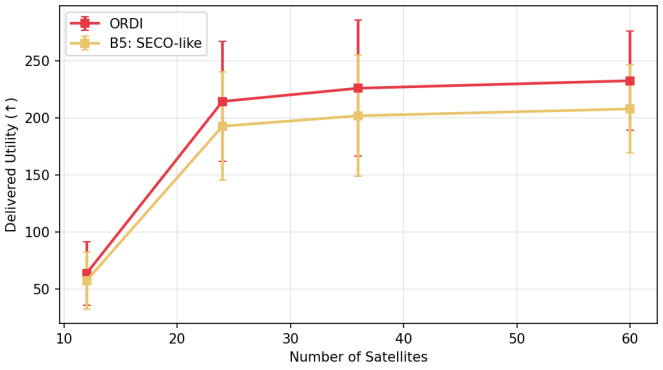

Fig. 4. Scalability with constellation size.

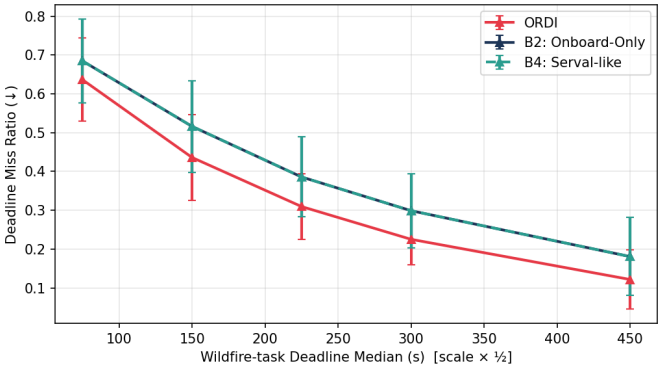

Fig. 5. Deadline tightness sweep.

in place of the RC model. Sentinel-2 acquisitions [14] are also supported as an alternative task source. The ordering established in Table I holds: ORDI attains the best delivered utility and realized objective at 0.87 replicas/tile and 19.3% misses, while full replication reaches a marginally lower 16.7% miss ratio only by paying 1.9× the replicas and energy, and non-distributed direct downlink misses 48%. The values reported throughout this paper (Table I) are thus corroborated by, and should be read as representative of, this real-data configuration.

VII. Discussion and Limitations

Fidelity. Our evaluation is simulation calibrated with in-orbit measurements, not a flight experiment. The compute profiles for the four workloads are benchmark-derived; we plan hardware-in-the-loop measurement on Jetson-class accelerators to replace them and to validate the thermal-throttling model against device behavior. **Reliability parameters.** Link success probabilities are literature-typical constants; weather-dependent downlink models would sharpen the adverse cases.

Coordination. ORDI assumes the source satellite has epoch-fresh constellation state; quantifying staleness tolerance, and a decentralized variant, is future work. **Coding.** We compare against an MDS-style baseline but do not explore coded *partial* redundancy (fractional parity tiles), which may dominate replication for large tile counts.

Deployment. ORDI admits two deployment models. Contact windows are deterministic given ephemerides, so a ground operator can precompute epoch graphs and uplink them with tasking; only the per-epoch assignment loop, sub-second at 60 satellites and dominated by dense array lookups after the routing sweeps, must run onboard. Alternatively, the full

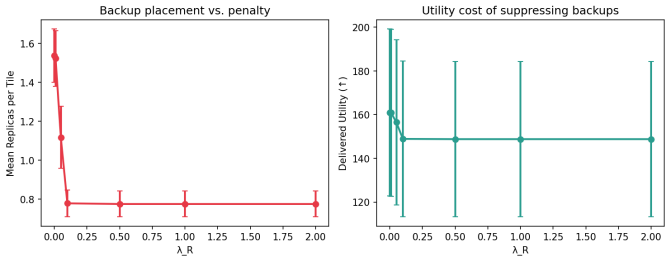

Fig. 6. Replication penalty sweep: λ_R is an effective redundancy dial.

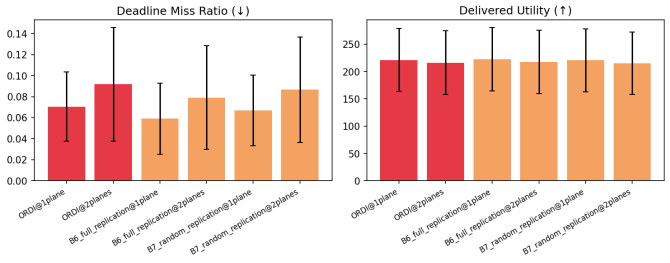

Fig. 7. Correlated plane-wide outages: plane-disjoint backups match full replication at lower redundancy.

scheduler can run on the source satellite, since the Tiansuan logs show the assignment workload is negligible next to a single tile inference. Replanning is local by construction: it touches only tiles whose replicas failed, so a recovery decision does not require re-coordinating the constellation. **Generality.** Nothing in the design is specific to the Walker pattern or the two-station ground segment; the contact graph abstracts both. Denser ground segments shrink ℓ^{down} and shift the bottleneck toward compute, which strengthens the case for distribution while leaving the redundancy calculus unchanged.

VIII. Related Work

Orbital edge computing. Denby and Lucia introduced orbital edge computing and pipelined onboard processing [1], and Kodan addressed onboard model specialization under compute constraints [3]. Serval schedules near-real-time EO queries via priority bifurcation on a single satellite's resources [2]. The Tiansuan constellation reported the first systematic in-orbit measurements of COTS AI payloads [4], which we use directly for calibration. ORDI differs in distributing single-task inference across satellites with explicit, priced fault tolerance.

Multi-satellite computation placement. Placement work such as SECO [5] optimizes latency or energy across satellites but assumes scheduled resources survive; we show this assumption costs 3.3× in miss ratio at realistic fault rates.

Redundancy in distributed systems. Tail-tolerant duplication [6] and coded computation [7], [8] target datacenters with cheap bandwidth and independent failures. ORDI adapts the marginal-gain view of redundancy to a regime where every replica is energy- and contact-priced and failures are structurally correlated.

Satellite scheduling. Classical EO scheduling allocates imaging and downlink windows [9]; ORDI sits downstream, scheduling the computation between capture and delivery.

IX. Conclusion

ORDI treats redundancy as a priced, orbit-aware resource rather than a blanket policy. One well-placed, fault-disjoint backup, admitted only when its marginal reliability gain justifies its energy and bandwidth cost, recovers nearly all of full replication's fault tolerance, including against correlated plane-wide outages, at 30% fewer replicas and 32% less energy, while a greedy rolling-horizon scheduler matches an ILP reference at scale-friendly cost. Calibration with public in-orbit COTS measurements grounds these conclusions in flight-measured power, compute, and battery behavior.

We see three immediate extensions. First, hardware-in-the-loop calibration: replacing benchmark-derived workload profiles with per-power-mode latency and energy measurements on Jetson-class flight-representative accelerators, validating the thermal-throttling model against observed frequency scaling. Second, coded partial redundancy: fractional parity tiles between one replica and two may dominate both on the energy-reliability frontier for large tile grids. Third, decentralization: bounding how stale a satellite's view of constellation state can be before selective redundancy loses its advantage, and gossiping only the state that the marginal-gain test actually consumes. Code, declarative fault specifications, and all experiment configurations are available for reproduction.

References

[1] B. Denby and B. Lucia, "Orbital edge computing: Nanosatellite constellations as a new class of computer system," in *Proc. ACM ASPLOS*, 2020.

[2] B. Tao, O. Chabra, I. Janveja, I. Gupta, and D. Vasisht, "Known knowns and unknowns: Near-realtime earth observation via query bifurcation in serval," in *Proc. USENIX NSDI*, 2024.

[3] B. Denby, K. Chintalapudi, R. Chandra, B. Lucia, and S. Noghabi, "Kodan: Addressing the computational bottleneck in space," in *Proc. ACM ASPLOS*, 2023.

[4] R. Xing, M. Xu, A. Zhou, Q. Li, Y. Zhang, F. Qian, and S. Wang, "Deciphering the enigma of satellite computing with COTS devices: Measurement and analysis," in *Proc. ACM MobiCom*, 2024.

[5] Z. Zhai, L. Zeng, T. Ouyang, S. Yu, Q. Huang, and X. Chen, "SECO: Multi-satellite edge computing enabled wide-area and real-time earth observation missions," in *Proc. IEEE INFOCOM*, 2024.

[6] J. Dean and L. A. Barroso, "The tail at scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[7] K. Lee, M. Lam, R. Pedarsani, D. Papailiopoulos, and K. Ramchandran, "Speeding up distributed machine learning using codes," *IEEE Transactions on Information Theory*, vol. 64, no. 3, pp. 1514–1529, 2018.

[8] X. Liu, C. Huang, and M. Tang, "CoCoI: Distributed coded inference system for straggler mitigation," in *Proc. WiOpt*, 2025.

[9] M. Lemaître, G. Verfaillie, F. Jouhaud, J.-M. Lachiver, and N. Bataille, "Selecting and scheduling observations of agile satellites," *Aerospace Science and Technology*, vol. 6, no. 5, pp. 367–381, 2002.

[10] NASA Small Spacecraft Systems Virtual Institute, "State-of-the-art of small spacecraft technology: Communications," https://www.nasa.gov/smallsat-institute/sst-soa/soa-communications/, 2026, accessed July 10, 2026.

[11] T. S. Kelso, "CelesTrak: Current GP element sets," https://celestrak.org/NORAD/elements/, accessed July 10, 2026.

[12] N. FIRMS, "Fire information for resource management system (firms): Active fire data," https://firms.modaps.eosdis.nasa.gov/, vIIRS S-NPP 375 m active-fire product; accessed July 10, 2026.

[13] OroraTech, "Wildfire constellation: Sub-10-minute thermal-anomaly alerting," https://www.eoportal.org/satellite-missions/ororatech-wildfire-constellation, accessed July 10, 2026.
[14] Element 84, "Earth search: STAC API for Sentinel-2 on AWS open data," https://earth-search.aws.element84.com/v1, accessed July 10, 2026.

APPENDIX A
FULL SIMULATION PARAMETERS

Table III lists every parameter of the simulator, grouped by subsystem. Satellite hardware parameters are not assumed: they are parsed at runtime from the public Tiansuan BUPT-1 Atlas 200DK in-orbit logs [4] (inference throughput from the 4-thread image-classification log, power from telemetry, battery capacity from published charge curves).

TABLE III
COMPLETE SIMULATION PARAMETERS.

Group	Parameter	Value
Constellation	Pattern	Walker, 6 planes × 6 sats
	Altitude / period	550 km / 5760 s
	Horizon	17,280 s (288 epochs of 60 s)
	Ground stations	Fairbanks, Greenwich
	GS min. elevation	5° (25° in core comp.)
	ISL rate	200 Mb/s
Tasks	Arrival process	Poisson, 6 tasks/orbit
	FOV gating	600 km radius, 100 targets
	Deadlines	log-normal, σ=0.6
	Tile size	128×128×3 uint8
Scheduler	λ_E (energy)	10^{-5} utility/J
	λ_C (comm)	10^{-12} utility/wt-bit
	λ_R (replication)	0.05 utility/replica
	α (freshness)	0.002 s^{-1}
	Replica cap $r_{\max}$	2 (primary + 1 backup)
Reliability	π_{isl}	0.97
	π_{down}	0.92 (0.70 adverse)
	π_{node}	0.98
	Straggler trigger	1.5× expected latency
Satellite (measured, Atlas 200DK BUPT-1 [4])	Compute rate	9.67 GFLOP/s
	Idle power	7.78 W
	Compute power (incr.)	2.41 W
	Comms power (incr.)	2.49 W
	Battery capacity	118.87 Wh
	Battery floor	15%
	Solar harvest (avg.)	14.43 W
	Thermal limit / max obs.	80°C / 57°C
	Thermal RC constant	300 s

TABLE IV
PER-TILE WORKLOAD PROFILES (FOUR EO INFERENCE TYPES). D = MEDIAN DEADLINE AT REFERENCE SCALE.

Type	Model	In	Out	GFLOP	u	D (s)
Wildfire	MobileNetV2	48 kB	1 kB	0.3	1.0	300
Ship	YOLOv5-n	48 kB	5 kB	0.9	0.8	600
Cloud	U-Net (lt.)	48 kB	50 kB	1.2	0.5	1200
Change	Siamese CNN	96 kB	10 kB	1.8	0.9	480

APPENDIX B
COMPLETE CORE-COMPARISON RESULTS

Table II reports all ten metrics of the core comparison, mean over 60 seeds (Table I shows the headline columns). Note B6–B8 lose 9–14 objective points to the replication and congestion penalties of (3) that ORDI largely avoids.

APPENDIX C
FAULT INJECTION DETAILS

Faults are declarative specifications resolved against the built constellation at runtime. The seven classes and their mechanics: *ISL disruption* removes a specific ISL edge from $E(t)$ for the fault duration; *orbital-plane outage* sets A_i=0 for every satellite in one plane; *helper failure* sets A_i=0 for one satellite, triggering replanning of its assigned tiles; *straggler* scales the helper's C_i by $0.1\times$ during execution; *ground-contact miss* removes a downlink window; *battery shortage* drains B_i below the 15% floor, forcing A_i=0 until recharge; *thermal throttling* raises Θ_i past the limit, reducing C_i to 25%.

The fault-intensity sweep draws, per epoch with probability equal to the fault rate, one fault uniformly from {helper failure, straggler, battery shortage, thermal throttle, ISL disruption} on a random satellite, with durations of 1–3 epochs. The correlated-failure experiment instead disables whole planes at mid-horizon, with backups constrained to plane-disjoint placement.

APPENDIX D
PER-EXPERIMENT CONFIGURATIONS

Table V gives the exact configuration of every experiment. All sweeps hold the Section VI-A defaults except the swept variable; matched seeds are used across algorithms within each sweep point.

TABLE V
EXPERIMENT CONFIGURATIONS.

Exp.	Swept variable	Algorithms	Seeds
Core (VI-B)	none (25° GS elev.)	all 9	60
Fault types	7 fault classes	ORDI	8
Intensity (VI-D)	rate $\in \{0, .05, .1, .2, .35, .5\}$	ORDI, B5, B6	24
Scalability	$N \in \{12, 24, 36, 60\}$	ORDI, B5	8
Deadline	scale $\in \{150..900\}$ s	ORDI, B2, B4	8
λ_R sweep	$\{0, .01, .05, .1, .5, 1, 2\}$	ORDI	8
Correlated (VI-H)	1–2 plane outages	ORDI, B6, B7	8
ILP gap (VI-I)	none (12 sats, 3/orbit)	greedy, ILP	1
Real (VI-J)	Planet TLE + FIRMS + telemetry	all 9	8

TABLE II

COMPLETE CORE-COMPARISON RESULTS, MEAN OVER 60 SEEDS. CVG. = PARTIAL COVERAGE; REC. = RECOVERY LATENCY; UTIL% = HELPER UTILIZATION; OBJ. = REALIZED OBJECTIVE; REPL. = MEAN REPLICAS PER TILE.

Algorithm	Miss	Utility	Cvg.	Rec. (s)	ISL (Mb)	Downlink (Mb)	Energy (J)	Util%	Obj.	Repl.
B1 direct	0.951	8.0	0.049	305.6	0	8.27	0.41	0	8.0	0
B2 onboard	0.273	137.2	0.727	222.6	0	40.19	211.5	0.003	137.2	0.73
B3 compress	0.273	140.0	0.727	222.6	0	18.74	21.9	0.003	140.0	0.73
B4 Serval-like	0.273	137.1	0.727	223.0	0	40.19	211.5	0.003	137.1	0.73
B5 SECO-like	0.217	145.9	0.783	192.7	135.3	41.74	228.0	0.004	145.9	0.78
B6 full repl.	0.217	156.4	0.783	229.9	269.3	41.74	452.0	0.007	142.7	1.56
B7 random repl.	0.217	145.9	0.783	268.0	349.7	41.74	455.9	0.007	132.1	1.57
B8 CoCoI-like	0.217	156.5	0.783	229.8	270.8	41.74	453.9	0.007	142.7	1.57
ORDI	**0.217**	**160.1**	**0.783**	**191.7**	**77.6**	**41.74**	**309.4**	**0.005**	**154.5**	**1.10**